

Lab Two: Exploring Image Data

Group Members: David Guo, Zi Ling

1. Business Understanding

Garbage classification is a key process in waste management. Clear classification will reduce contamination within the waste, improve the waste recycling and reduce landfill.

Although public spaces and homes already have bins for recyclable and non-recyclable waste, there are three problems that can't be solved:

- 1. Misclassification always happens because many people lack knowledge or just ignore the garbage type, and contaminate the recycling garbage.
- 2. Classification is not detailed enough, plastic and cardboard are mixed in the recycling classification, so refining the classification is needed.
- 3. Refining classification usually requires a lot of human resources, which means the cost is high.

Therefore, automatic garbage classification is essential for our daily life, related companies, and the environment of our planet.

To achieve automatic classification, the first and most important thing is to recognize garbage and build a predictive model to classify it into the right categories.

To support this analysis, we use the Garbage Classification Dataset from Kaggle. This dataset has about 1,3901 images in six categories, including cardboard, glass, metal, paper, plastic, and trash. The distribution is fairly even, each category has 2000-2500 images, with trash and glass having the most images of 18%, and metal has the fewest images of 15%. All images are standardized, resized to 256x256 pixels, in RGB format, and cleaned of duplicates for quality and consistency.

This dataset is practical and useful for building a predictive model that classifies garbage into the right categories. With this model, the garbage classification companies and the waste management department can build automatic garbage bins, or assembly line machines, to improve the efficiency, correctness of garbage classification, and reduce costs.

To achieve a useful model, the accuracy of the model needs to be higher than 90%, because mistakes can cause safety issues, especially for glass and plastic. However, it's not a live critical tool, and the refining classification should still be in use, so we don't need to achieve a 99% level of accuracy, because it will cost more resources.

=====

Dataset URL: <https://www.kaggle.com/datasets/zlatan599/garbage-dataset-classification/data>

Questions of Interest: Predict the garbage category.

2. Data Preparation

```
In [1]: # Code from MachineLearningNotebook
# Asked GPT to help fix the directory issue
import numpy as np
from PIL import Image
import os
import warnings
warnings.filterwarnings("ignore")

base_dir = os.path.abspath("./images")
folder_names = os.listdir(base_dir)

image_data_gray = []
image_label = []

for folder_name in folder_names:

    if folder_name == ".DS_Store": #skip system file on Mac
        continue

    folder_path = os.path.join(base_dir, folder_name)

    for image_name in os.listdir(folder_path):
        image_path = os.path.join(folder_path, image_name)
        img = Image.open(image_path)
        img = img.resize((64, 64))

        img_gray = img.convert('L') # Convert to grayscale to reduce computation co
        img_array = np.array(img_gray)

        flatten_image = img_array.flatten() # Creating a 1D vector
        image_data_gray.append(flatten_image)
        image_label.append(folder_name)

image_data_gray = np.array(image_data_gray)
image_label = np.array(image_label)
```

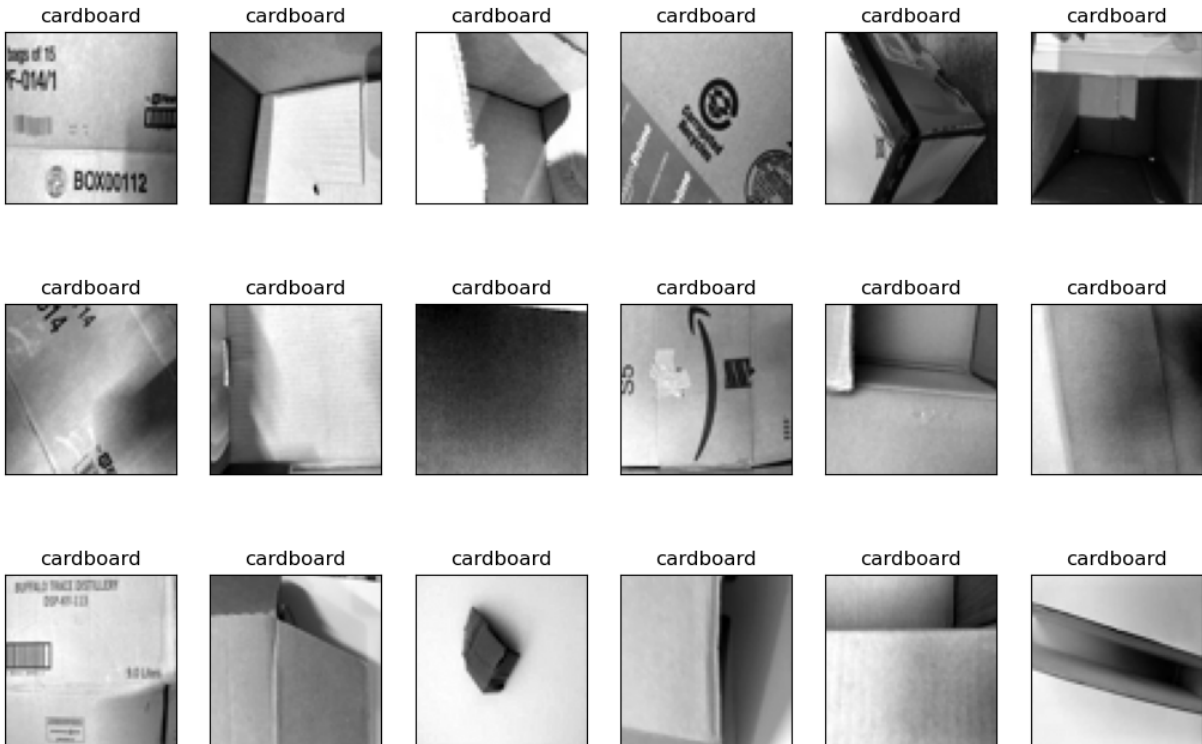
```
In [2]: ##This code block was taken from Eric Larson code notebook
import matplotlib.pyplot as plt

# a helper plotting function
def plot_gallery(images, titles, h, w, n_row=3, n_col=6):
    """Helper function to plot a gallery of portraits"""
    plt.figure(figsize=(1.7 * n_col, 2.3 * n_row))
    plt.subplots_adjust(bottom=0, left=.01, right=.99, top=.90, hspace=.35)

    for i in range(n_row * n_col):
```

```
plt.subplot(n_row, n_col, i + 1)
plt.imshow(images[i].reshape((h, w)), cmap='gray') # Use 'gray' colormap f
plt.title(titles[i], size=12)
plt.xticks(())
plt.yticks(())

plot_gallery(image_data_gray, image_label, 64, 64)
```



3. Data Reduction

In [3]: *# precheck the data before reduction, code from MachineLearningNotebook*

```
# define X and y
X = image_data_gray
y = image_label

n_samples, n_features = X.shape
n_classes = len(np.unique(y))

print(np.sum(~np.isfinite(X))) # check if there is any NaN or Inf value in X
print("n_samples: {}".format(n_samples))
print("n_features: {}".format(n_features)) # 64*64=4096
print("n_classes: {}".format(n_classes))
```

```
0
n_samples: 13901
n_features: 4096
n_classes: 6
```

The image data is clean, with 13901 samples, 4096 features, and 6 classes, which meets our expectations.

So we can continue to reduce the dimension.

3.1 Principal Components Analysis (PCA)

```
In [4]: # PCA (code from MachineLearningNotebooks)
from sklearn.decomposition import PCA

n_components = 300 # assume we want to keep 300 components
print("Extracting the top %d components from %d samples" % (
    n_components, X.shape[0]))

pca = PCA(n_components=n_components)

# measure runtime
%time pca.fit(X.copy())

# def plot
def plot_explained_variance(pca):
    import plotly
    from plotly.graph_objs import Bar, Line
    from plotly.graph_objs import Scatter, Layout
    from plotly.graph_objs.scatter import Marker
    from plotly.graph_objs.layout import XAxis, YAxis
    plotly.offline.init_notebook_mode()

    explained_var = pca.explained_variance_ratio_
    cum_var_exp = np.cumsum(explained_var)

    plotly.offline.iplot({
        "data": [Bar(y=explained_var, name='individual explained variance'),
                 Scatter(y=cum_var_exp, name='cumulative explained variance')
                ],
        "layout": Layout(xaxis=XAxis(title='Principal components'), yaxis=YAxis(tit
    })

# plot explained variance
plot_explained_variance(pca)
```

```
Extracting the top 300 components from 13901 samples
CPU times: total: 33.5 s
Wall time: 4.7 s
```



We performed PCA with 300 components and created the explained variance plot.

The plot shows:

1. 19 components have already captured 80% of the variance.
2. 90 components have already captured 90% of the variance.
3. After 200 components, the curve flattens and achieves 93.7% of the variance.

As we discussed in the business understanding section, we suppose the model accuracy is reached at least 90%, so the variance caption also needs to meet our expectation. According to the plot findings, 90 components seem to be enough. But we think 200 components is better in this case, because 90 components only reach our bottom line, we want a higher variance. And the curve flattens after 200 components, which means that after 200, more components will not bring more benefit.

Therefore, we think 200 components (dimensions) are required to adequately represent our image data.

3.2 Randomized Principle Components Analysis

```
In [5]: # randomized PCA, code from MachineLearningNotebooks

n_components = 300 #using the same number of components with PCA
print("Extracting the top %d components from %d samples" % (
    n_components, X.shape[0]))

rpca = PCA(n_components=n_components, svd_solver='randomized')

# measure runtime
%time rpca.fit(X.copy())

plot_explained_variance(rpca)
```

Extracting the top 300 components from 13901 samples

CPU times: total: 33.5 s

Wall time: 4.59 s



We performed Randomized PCA with 300 components (same as PCA) and created the explained variance plot.

The plot shows a similar result to PCA:

1. 19 components have already captured 80% of the variance.
2. 90 components have already captured 90% of the variance.

3. After 200 components, the curve flattens and achieves 93.7% of the variance.

Since the explained variance result of Randomized PCA is similar to PCA, our component choosing is the same, which is considering our business accuracy goal and performance goal, we choose 200 components (dimensions) as the required components to adequately represent our image data.

3.3 Methods Comparison

3.3.1 Explained Variance Comparison

According to the PCA and RPCA Explained Variance Comparison, they have the same result (93.7%) when we chose 200 components.

Therefore, we can't decide which method is better based on the explained variance.

3.3.2 Runtime Comparison

```
In [6]: # PCA and RPCA in 200 components (code from MachineLearningNotebooks)

from sklearn.decomposition import PCA
n_components = 200 # the final components we chose
print("Extracting the top %d components from %d samples using PCA" % (
    n_components, X.shape[0]))

pca = PCA(n_components=n_components)

# measure runtime
%time pca.fit(X.copy())

# randomized PCA
n_components = 200 #using the same number of components with PCA
print("Extracting the top %d components from %d samples using RPCA" % (
    n_components, X.shape[0]))
rpca = PCA(n_components=n_components, svd_solver='randomized')
%time rpca.fit(X.copy());
```

```
Extracting the top 200 components from 13901 samples using PCA
CPU times: total: 25.5 s
Wall time: 3.59 s
Extracting the top 200 components from 13901 samples using RPCA
CPU times: total: 25.2 s
Wall time: 3.48 s
```

We tried several times running PCA and RPCA, and the results shows that:

1. Wall time of PCA and RPCA are pretty close, the difference is less than one second.
2. RPCA is not always faster than PCA, sometime RPCA is slower.

Besides, the runtime is not the most critical thing when choosing a method

Therefore, we performed more comparisons.

3.3.3 Reconstruction Error

```
In [7]: # use root mean squared error for PCA and RPCA to compare them
# reference idea from https://stats.stackexchange.com/questions/194278/meaning-of-r
# debugged by GPT

from math import sqrt
from sklearn.metrics import mean_squared_error

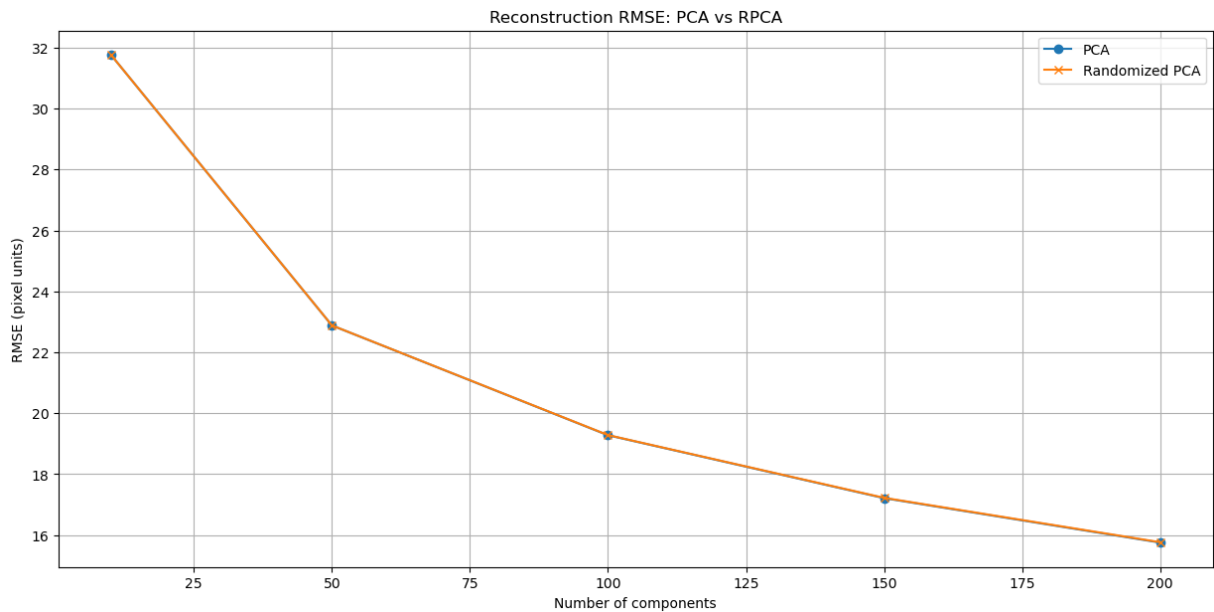
# function to calculate RMSE
def reconstruction_rmse(model, X):
    X_proj = model.transform(X) # project to lower dimension
    X_reconstr = model.inverse_transform(X_proj) # reconstruct back to original dim
    rmse = sqrt(mean_squared_error(X, X_reconstr)) # calculate RMSE
    return rmse

components_range = [10, 50, 100, 150, 200]
rmse_pca, rmse_rpca = [], []

for k in components_range:
    pca_k = PCA(n_components=k, svd_solver="full", random_state=0).fit(X)
    rpca_k = PCA(n_components=k, svd_solver="randomized", random_state=0).fit(X)

    rmse_pca.append(reconstruction_rmse(pca_k, X))
    rmse_rpca.append(reconstruction_rmse(rpca_k, X))

# plot RMSE
plt.figure(figsize=(15,7))
plt.plot(components_range, rmse_pca, marker='o', label="PCA")
plt.plot(components_range, rmse_rpca, marker='x', label="Randomized PCA")
plt.xlabel("Number of components")
plt.ylabel("RMSE (pixel units)")
plt.title("Reconstruction RMSE: PCA vs RPCA")
plt.legend()
plt.grid(True)
plt.show()
```

To achieve quantitative comparison, we first use the Reconstruct Error to compare the PCA and RPCA dimensionality reduction effect.

By calculating the Root Mean Squared Error (RMSE) between reduced-dimension images and the original image, we can see how different they are. High RMSE means the image after PCA or RPCA is much more different from the original image, vice versa.

According to the plot, the Root Mean Squared Error (RMSE) for PCA and RPCA is the same in our dataset, no matter the components number.

Therefore, we can't confirm which method is better by calculating RMSE based on our dataset.

3.3.4 KNN Accuracy Comparison

```
In [8]: # use KNN classification accuracy to compare PCA and RPCA
# debugged by GPT in accuracy calculation
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

# split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

components_range = [10, 50, 100, 150, 200]
acc_pca, acc_rpca = [], []

for k in components_range:
    # pca
    pca_k = PCA(n_components=k, svd_solver="full", random_state=0).fit(X_train)
    X_train_pca = pca_k.transform(X_train)
```

```

X_test_pca = pca_k.transform(X_test)

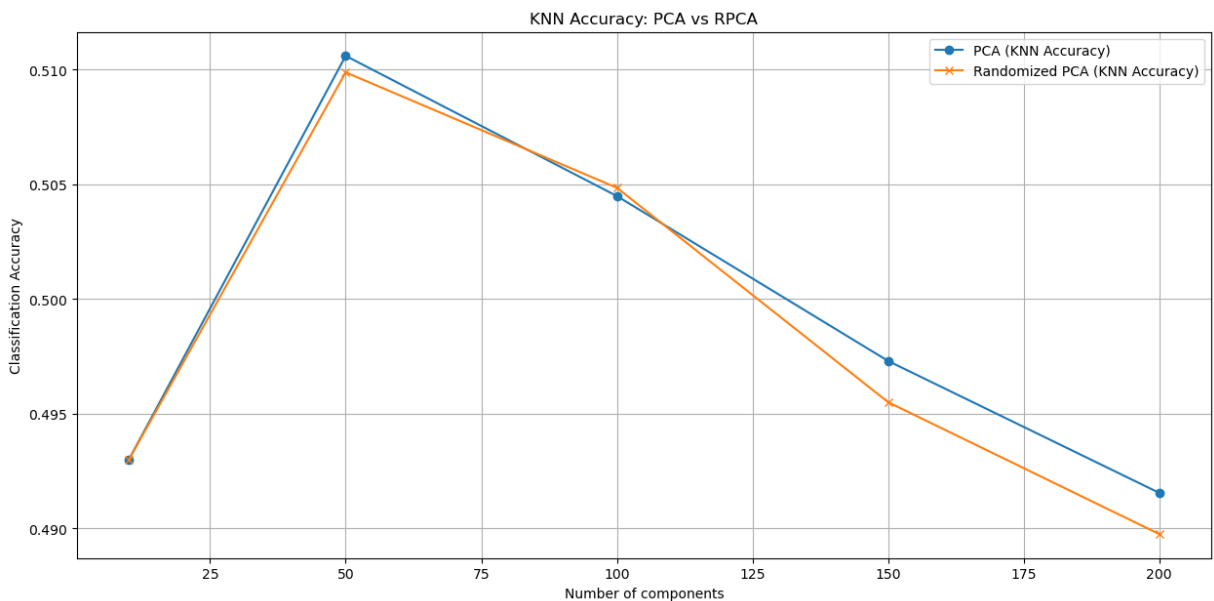
knn_pca = KNeighborsClassifier(n_neighbors=3) # KNN classifier
knn_pca.fit(X_train_pca, y_train)
acc_pca.append(np.mean(knn_pca.predict(X_test_pca) == y_test)) # calculate acc

# rpca
rpca_k = PCA(n_components=k, svd_solver="randomized", random_state=0).fit(X_train)
X_train_rpca = rpca_k.transform(X_train)
X_test_rpca = rpca_k.transform(X_test)

knn_rpca = KNeighborsClassifier(n_neighbors=3)
knn_rpca.fit(X_train_rpca, y_train)
acc_rpca.append(np.mean(knn_rpca.predict(X_test_rpca) == y_test))

# plot accuracy
plt.figure(figsize=(15,7))
plt.plot(components_range, acc_pca, marker='o', label="PCA (KNN Accuracy)")
plt.plot(components_range, acc_rpca, marker='x', label="Randomized PCA (KNN Accuracy)")
plt.xlabel("Number of components")
plt.ylabel("Classification Accuracy")
plt.title("KNN Accuracy: PCA vs RPCA")
plt.legend()
plt.grid(True)
plt.show()

```



To dig deeper into quantitative analysis, we split the dataset into training and testing subsets, using KNN to train the model and predict the garbage classification on the testing data, then calculate the accuracy of the predictive result.

In this way, we hope that we can confirm that PCA or RPCA is better.

According to the result plot, the classification accuracy is pretty close between PCA and RPCA, both of them have the accuracy about 50%-54%, which means both method are not well for our business goal. Although the RPCA is slightly better than PCA, the difference between them is smaller than 0.003. The difference is too small, so it can't show that RPCA is

surely better than PCA.

Conclusion in PCA and RPCA Comparison

In the comparison phase, we performed several methods to find which one is better:

1. For explained variance, PCA and RPCA have the same variance ratio when the number of components is the same.
2. For run time, RPCA is slightly faster in wall time than PCA, but not always faster.
3. For the reconstruction error, PCA and RPCA have the same RMSE when the number of components is the same.
4. For the KNN classification accuracy, RPCA is slightly higher than PCA, but the difference is only 0.002, which is not sufficient to decide RPCA is surely better than PCA.

Therefore, we think PCA and RPCA have almost the same performance, so we don't prefer anyone of them.

(But if we have to choose one, we choose RPCA, at least it has a 0.002 higher accuracy.)

3.4 Feature Extraction Using DAISY

```
In [9]: # code from MachineLearningNotebook
from skimage.feature import daisy
from skimage.io import imshow

daisy_features_list = []

for i, single_image in enumerate(image_data_gray):
    #Need to reshape the image for daisy
    single_image_reshape = single_image.reshape(64, 64)

    daisy_features = daisy(single_image_reshape,
                           step=4,
                           radius=10,
                           rings=2,
                           histograms=6,
                           orientations=8,
                           )

    flattened_daisy = daisy_features.flatten()
    daisy_features_list.append(flattened_daisy)

daisy_features_array = np.array(daisy_features_list)
print(f"Final DAISY features shape: {daisy_features_array.shape}")
```

Final DAISY features shape: (13901, 12584)

After performing feature extraction using Daisy with a step of 4, radius of 10, ring of 2, histogram of 6, and orientation of 8, we got a shape of 13901 and 12584. The value of 13901 is expected as that's the total number of pictures from our dataset, and the value of 12584 indicates that Daisy was able to extra a lot of features from the photos due to smaller step

size, and radius.

This indicates that Daisy was able to capture a lot of local gradients across each image, however it does come with an issue of overfitting since the dimensional is high, which we will further explore in the following section.

3.5 Finding if the extracted feature is useful for the creating a classifier

In [10]: *#Encoding the labels to numeric*

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

numeric_label = label_encoder.fit_transform(image_label)

print(numeric_label)
```

[0 0 0 ... 5 5 5]

In [11]: *# code write by David*

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score

daisy_train, daisy_test, label_train, label_test = train_test_split(daisy_features_

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(daisy_train, label_train)
label_pred = knn.predict(daisy_test)

print("Accuracy: ", accuracy_score(label_test, label_pred))
print(classification_report(label_test, label_pred, target_names=label_encoder.clas
```

Accuracy: 0.5026968716289104

	precision	recall	f1-score	support
cardboard	0.43	0.55	0.49	449
glass	0.60	0.50	0.54	506
metal	0.44	0.63	0.52	389
paper	0.52	0.39	0.45	478
plastic	0.64	0.44	0.52	479
trash	0.47	0.53	0.50	480
accuracy			0.50	2781
macro avg	0.52	0.51	0.50	2781
weighted avg	0.52	0.50	0.50	2781

Looking at the result of KNN classifier, we did not get a good result with a K of 3. The overall accuracy was 50%, and looking at each class precision only the glass and plastic had good value. Indicating that knn was able to classify glass, and plastic better than the other group.

This could be because we resize the image to a 64x64 pixel and doing so we might have lost some details that could help classify the image. Additionally, the daisy parameters may not have been optimal for this pixel size, which resulted in loss of features that distinguish different categories.

As such for the feature extraction method does not look promising if we keep it as a 64x64 pixel size. It could perform better if we had more pixels so that we can extract more useful features, or tuning the parameter of daisy to extract the feature but keeping the parameter of Knn and the daisy the same as what we have is not the best.

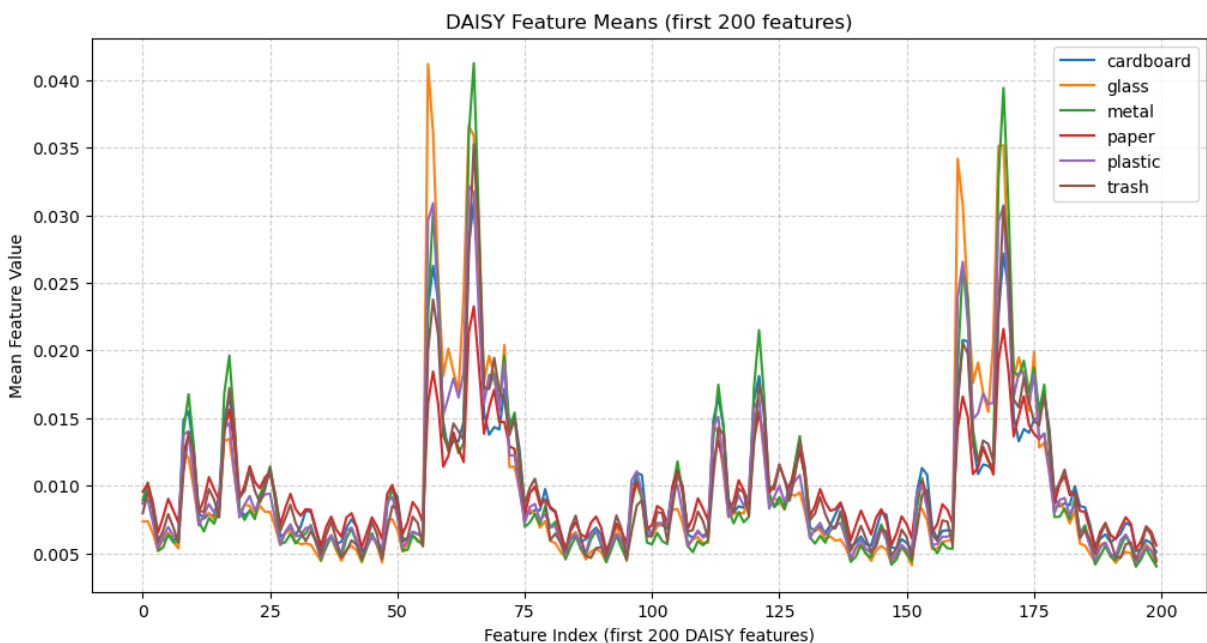
```
In [14]: # Write by David and used gpt to debug
import matplotlib.pyplot as plt

means = []

for class_names in np.unique(numeric_label):
    class_feature = daisy_features_array[numeric_label == class_names]
    class_mean = class_feature.mean(axis=0)
    means.append(class_mean)

means = np.array(means)

plt.figure(figsize=(12,6))
for i, mean_vec in enumerate(means[:6]):# changed from 5 to 6 to include all classes
    plt.plot(mean_vec[:200], label=f"{label_encoder.classes_[i]}")
plt.xlabel("Feature Index (first 200 DAISY features)")
plt.ylabel("Mean Feature Value")
plt.title("DAISY Feature Means (first 200 features)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



Graph Analysis.

From this result, we can see that Daisy was able to extract more information from photos labels such as glass or metal, and other labels showed some overlap. This is because plastic, paper, and cardboard have similar textures to one another, which could cause daisy to not extract as much information out compared to glass and metal with their reflective, and uneven edges.

Conclusion

PCA, RPCA, and Daisy all showed poor accuracy, which were around 50%. As such, PCA, RPCA, and Daisy are not suitable to use as data dimensional reduction tools for this dataset. We could tune the result, or use different algorithms, but in our opinion the result would be slighter, better or worse. This is due to the dataset being difficult to work with since we reduce the pixel size to 64x64 image for our computers to handle in a reasonable time, along with not tuning the parameters to get the best performance.

In conclusion, we do not prefer PCA, and RPCA as they achieved almost the same results. In addition, Daisy didn't show any promise either as such, this dataset is difficult to work with and would need further time to get better results to achieve higher accuracy.

4. Exceptional Work

```
In [13]: # Wrote by David and used gpt to handle repeated images being selected between loop
from sklearn.metrics import pairwise_distances_argmin_min

def compute_daisy_descriptors(img, step, radius):
    side = int(np.sqrt(img.shape[0]))
    img = img.reshape(side, side)

    desc = daisy(
        img,
        step=step,
        radius=radius,
        rings=2,
        histograms=6,
        orientations=8,
    )

    return desc.reshape(-1, desc.shape[-1])

def key_point_matching(img1, img2, threshold=0.15):
    daisy1, daisy2 = compute_daisy_descriptors(img1, 4, 10), compute_daisy_descript
    _, distance = pairwise_distances_argmin_min(daisy1, daisy2)
    matches = np.sum(distance < threshold)
    return matches

within_scores = []
between_scores = []
```

```
unique_labels = np.unique(numeric_label)

within_used = set()
between_used = set()

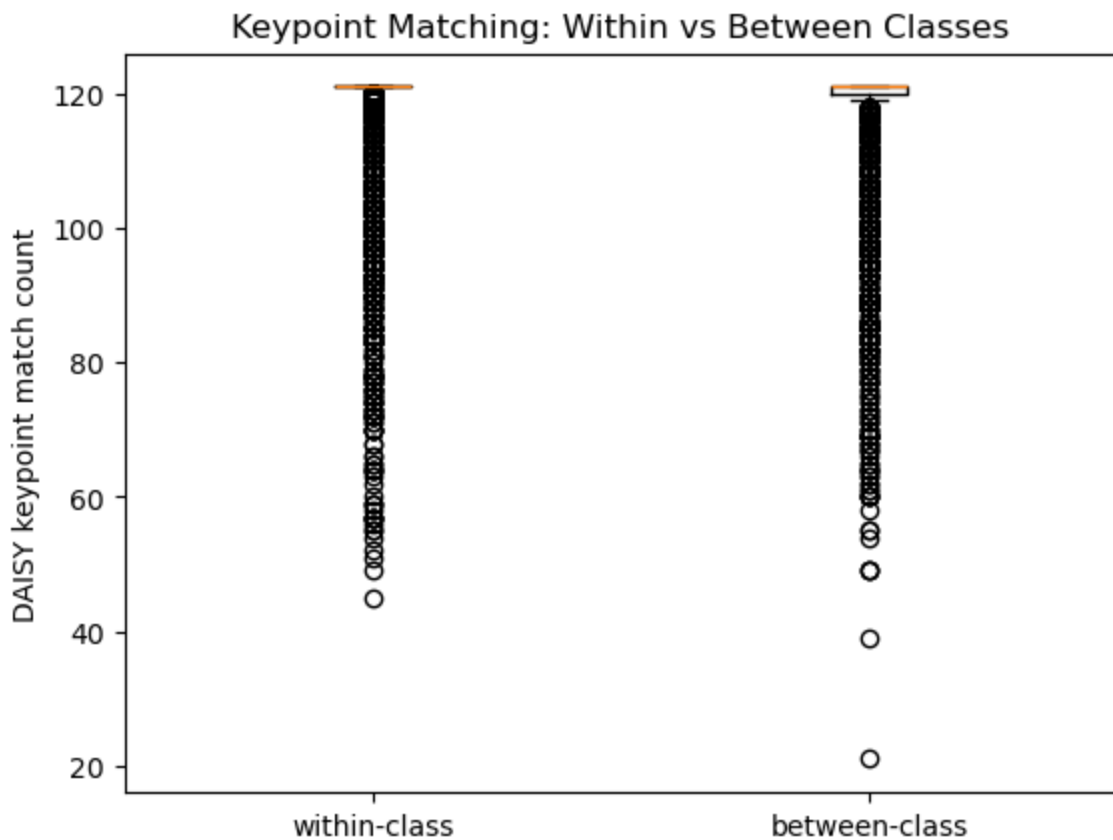
for _ in range(5000):
    random_class = np.random.choice(unique_labels)
    index = np.where(numeric_label == random_class)[0]
    i, j = np.random.choice(index, 2, replace=False)

    pair = tuple(sorted((i, j)))
    if pair not in within_used:
        within_used.add(pair)
        within_scores.append(key_point_matching(image_data_gray[i], image_data_gray[j]))

    random_label_1, random_label_2 = np.random.choice(unique_labels, 2, replace=False)
    i = np.random.choice(np.where(numeric_label == random_label_1)[0])
    j = np.random.choice(np.where(numeric_label == random_label_2)[0])

    pair = tuple(sorted((i, j)))
    if pair not in between_used:
        between_used.add(pair)
        between_scores.append(key_point_matching(image_data_gray[i], image_data_gray[j]))

plt.boxplot([within_scores, between_scores], labels=["within-class", "between-class"])
plt.ylabel("DAISY keypoint match count")
plt.title("Keypoint Matching: Within vs Between Classes")
plt.show()
```



Boxplot analysis

We first extract features using daisy, and from the extracted feature we then calculate the pairwise distance between the two extracted daisy and if we calculate the distance is less than 0.15, we would be considered as match. From there, we then run a loop 5000, and each time we ensure that we pick unique images for comparison by using set to keep track of the images.

From the result of the boxplot, we can see that the distribution between within class, and between class are almost the same. This indicates that daisy descriptors did not provide strong matches between key points in the dataset. This could be due to various reasons from the dataset to the parameters we choose when computing daisy. As such, we can conclude that the key matching did not perform well on this dataset. Increasing the pixel size could provide better results, but it is expensive to run daisy on the entire dataset.